Lab08 - Accelerometer

1. Il bus e il protocollo I2C verso l'accelerometro

L'accelerometro è uno slave I2C. Ha un indirizzo a 7 bit che dipende dal chip:

- **LIS2DE = 0b0101000 (0x28)**
- **LIS2DW = 0b0011000 (0x18)**

Attenzione: le funzioni HAL (HAL_I2C_Master_Transmit, ecc.) vogliono l'indirizzo già shiftato a sinistra di 1 bit (DevAddress = addr << 1), quindi in codice userai 0x28<<1 = 0x50 oppure 0x18<<1 = 0x30 (a seconda del chip).

Come si identifica il chip:

provi a fare una `HAL_I2C_Master_Transmit` verso uno dei due indirizzi con un buffer vuoto o un solo byte; se il dispositivo è presente risponde con ACK e la funzione ritorna `HAL_OK`, altrimenti (NACK) ritorna `HAL_ERROR`. Così, senza nemmeno leggere WHO_AM_I, sai già se hai il LIS2DE o il LIS2DW collegato.

Se invece vuoi confermarlo leggendo il registro **WHO_AM_I (0x0F)**:

- LIS2DE → valore fisso 0x33
- LIS2DW → valore fisso 0x44

Lettura multipla con auto-increment

Qui i due chip si comportano *diversamente*:

- **LIS2DE**: l'auto-increment è nel bit più significativo del sub-address stesso. Devi impostare il MSB (bit 7) del sub-address a 1 quando scrivi l'indirizzo del primo registro da leggere; il dispositivo poi incrementa automaticamente l'indirizzo interno ad ogni byte letto. Quindi se vuoi leggere OUT_X (0x29), OUT_Y (0x2B), OUT_Z (0x2D) in sequenza, il sub-address da inviare è 0x29 | 0x80 = 0xA9.
- **LIS2DW**: l'auto-increment è invece controllato dal bit **IF_ADD_INC** in **CTRL2 (0x21)**, che di default è già a 1 (abilitato). Con questo bit attivo basta scrivere il sub-address "normale" (senza settare alcun MSB) e il dispositivo incrementa da solo l'indirizzo ad ogni byte successivo.

2. Mappa dei registri utili (entrambi i chip)

Funzione	LIS2DE	LIS2DW
WHO_AM_I	0x0F → 0x33	0x0F → 0x44
Registro di controllo principale (ODR, enable assi)	CTRL_REG1 (0x20)	CTRL1 (0x20)
Full scale, filtri	CTRL_REG4 (0x23)	CTRL6 (0x25)
Dati uscita X	OUT_X (0x29) — 1 registro, 8 bit	OUT_X_L (0x28) + OUT_X_H (0x29) — 16 bit
Dati uscita Y	OUT_Y (0x2B)	OUT_Y_L (0x2A) + OUT_Y_H (0x2B)
Dati uscita Z	OUT_Z (0x2D)	OUT_Z_L (0x2C) + OUT_Z_H (0x2D)

Questa differenza è cruciale per rispondere alla domanda del **Project 01c** "quanti byte devi trasferire?": con **LIS2DE** ti servono **3 byte** (un byte per asse), con **LIS2DW** te ne servono **6 byte** (LSB+MSB per ciascun asse).

3. Configurazione richiesta dall'esercizio

L'obiettivo è: modalità normale, tutti e 3 gli assi attivi, ODR = 1 Hz, nessun filtro high-pass, full scale ±2g.

LIS2DE — CTRL_REG1 (0x20)

Bit: ODR3 ODR2 ODR1 ODR0 LPen Zen Yen Xen

- **ODR[3:0] = 0001** → 1 Hz (vedi tabella data-rate: 0000=power-down, 0001=1 Hz, 0010=10 Hz, ecc.)
- **LPen = 0** → modalità normale (non low-power)
- **Zen=Yen=Xen = 1** → tutti gli assi abilitati

Table 24. CTRL_REG1 register

ODR3	ODR2	ODR1	ODR0	LPen	Zen	Yen	Xen

Table 25. CTRL_REG1 register description

ODR [3:0]	Data rate selection. Default value: 0000 (0000: Power-down mode; Others: Refer to *Table 26*)
LPen	Low-power mode enable. Default value: 0 (0: Normal mode, 1: Low-power mode)
Zen	Z-axis enable. Default value: 1 (0: Z-axis disabled; 1: Z-axis enabled)
Yen	Y-axis enable. Default value: 1 (0: Y-axis disabled; 1: Y-axis enabled)
Xen	X-axis enable. Default value: 1 (0: X-axis disabled; 1: X-axis enabled)

Table 26. Data rate configuration

ODR3	ODR2	ODR1	ODR0	Power mode selection
0	0	0	0	Power-down mode
0	0	0	1	Normal / Low-power mode (1 Hz)
0	0	1	0	Normal / Low-power mode (10 Hz)
0	0	1	1	Normal / Low-power mode (25 Hz)
0	1	0	0	Normal / Low-power mode (50 Hz)
0	1	0	1	Normal / Low-power mode (100 Hz)
0	1	1	0	Normal / Low-power mode (200 Hz)
0	1	1	1	Normal / Low-power mode (400 Hz)
1	0	0	0	Low-power mode (1.6 KHz)
1	0	0	1	Normal (1.344 kHz) / Low-power mode (5.376 kHz)

Quindi CTRL_REG1 = 0001 0111 = **0x17**.

Il filtro high-pass si disabilita di default: non serve toccare CTRL_REG2 (default 0x00, HPCLICK/HPIS1/HPIS2 tutti a 0 → filtro non applicato all'uscita/FIFO). Attenzione solo a FDS in CTRL_REG2: default 0 = "internal filter bypassed" cioè dati non filtrati, che è quello che vuoi.

CTRL_REG4 (0x23) — full scale

Bit: BDU - FS1 FS0 - ST1 ST0 SIM

- **FS[1:0] = 00** → ±2g (già il default, quindi non dovresti nemmeno bisogno di scriverci, ma è buona pratica farlo esplicitamente)

- BDU a 0 va bene per letture continue (se lo metti a 1, i registri si aggiornano solo dopo che hai letto sia MSB che LSB — utile più che altro sul LIS2DW che ha 2 registri per asse)

CTRL_REG4 = 0000 0000 = **0x00** (va bene il default).

Table 32. CTRL_REG4 register

BDU	-	FS1	FS0	-	ST1	ST0	SIM

Table 33. CTRL_REG4 register description

BDU	Block data update. Default value: 0 (0: continuous update. For linear acceleration data output this bit must be set to 0; 1: this bit must be set to 1 for temperature sensor reading only)
FS [1:0]	Full-scale selection. Default value: 00 (00: ±2*g*; 01: ±4*g*; 10: ±8*g*; 11: ±16*g*)
ST [1:0]	Self-test enable. Default value: 00 (00: self-test disabled; Other: See *Table 34*)
SIM	SPI serial interface mode selection. Default value: 0 (0: 4-wire interface; 1: 3-wire interface)

Table 34. Self-test mode configuration

ST1	ST0	Self-test mode
0	0	Normal mode
0	1	Self-test 0
1	0	Self-test 1
1	1	--

LIS2DW — CTRL1 (0x20)

Table 28. Control register 1

ODR3	ODR2	ODR1	ODR0	MODE1	MODE0	LP_MODE1	LP_MODE0

Table 29. Control register 1 description

ODR[3:0]	Output data rate and mode selection (see *Table 30*)
MODE[1:0]	Mode selection (see *Table 31*)
LP_MODE[1:0]	Low-power mode selection (see *Table 32*)

Qui non esiste un ODR "1 Hz" esatto come nel LIS2DE: la tabella Data Rate Configuration del LIS2DW parte da 0001 = 12.5/1.6 Hz (in Low-Power) — il valore più vicino a 1 Hz disponibile è quello **1.6 Hz in modalità Low-Power** (serve ODR[3:0]=0001 insieme a MODE=00 Low-Power e un LP_MODE a scelta, tipicamente 00=Low-Power Mode 1, 12 bit). Non c'è un vero e proprio "1 Hz" su questo chip: la scelta più sensata per rispettare la specifica è ODR3..0 = 0001, MODE=00 (Low-Power), LP_MODE=00.
Se invece vuoi restare in **High-Performance** (14 bit, più preciso) userai MODE=01, ma anche lì l'ODR minimo utile resta 12.5 Hz — quindi per avvicinarti davvero a "1 Hz" la strada del datasheet è la modalità Low-Power a 1.6 Hz.

CTRL6 (0x25) — full scale sul LIS2DW

Bit: BW_FILT1 BW_FILT0 FS1 FS0 FDS LOW_NOISE 0 0

- **FS[1:0] = 00** → ±2g
- **FDS = 0** → filtro passa-basso selezionato (cioè nessun high-pass sull'uscita, che è quello che vuoi)

Table 42. Control register 6

BW_FILT1	BW_FILT0	FS1	FS0	FDS	LOW_ NOISE	0	0

Table 43. Control register 6 description

BW_FILT[1:0]	Bandwidth selection (see *Table 44*)
FS[1:0]	Full-scale selection (see *Table 45*)
FDS	Filtered data type selection. Default value: 0 (0: low-pass filter path selected; 1: high-pass filter path selected)
LOW_NOISE	Low-noise configuration. (0: disabled; 1: enabled)

4. Conversione del dato grezzo in g

Il valore letto è in complemento a 2, e va moltiplicato per la sensibilità (che dipende da chip, full scale e — sul LIS2DW — anche dalla modalità/risoluzione):

LIS2DE

FS=±2g: sensibilità tipica **15.6 mg/digit**, dato a 8 bit già "left-justified" nel registro (cioè leggi il byte OUT_X come intero con segno a 8 bit e lo moltiplichi direttamente per 15.6 mg).

```
acc_x_mg = (int8_t)OUT_X * 15.6;    // in mg
acc_x_g  = acc_x_mg / 1000.0;       // in g
```

LIS2DW

FS=±2g: sensibilità **0.976 mg/digit in Low-Power Mode 1** (dato a 12 bit, giustificato a sinistra nei 16 bit — i due bit meno significativi di OUT_x_L sono forzati a 0), oppure **0.244 mg/digit** in High-Performance/altri Low-Power Mode (dato a 14 bit). Devi quindi prima ricostruire il valore a 16 bit:

```
int16_t raw = (OUT_X_H << 8) | OUT_X_L;    // 16 bit, complemento a 2
acc_x_mg = raw * sensibilità;                // 0.976 o 0.244 mg/digit a
seconda del modo
```

Con precisione richiesta di 0.01 g = 10 mg, entrambe le sensibilità (15.6 mg su LIS2DE, 0.976/0.244 mg su LIS2DW) sono più che sufficienti; basta arrotondare correttamente in stampa (%+.2f g) rispettando il segno.

5. Collegamento diretto con i tre sotto-progetti

- **01a** (polling + UART semplice): identifica il chip col trucco del return value di HAL_I2C_Master_Transmit, configura CTRL_REG1/CTRL1 e CTRL_REG4/CTRL6 come sopra, poi in loop leggi i tre registri (uno alla volta o con HAL_I2C_Mem_Read), converti e stampi via UART con HAL_Delay.
- **01b**: stessa logica di lettura/conversione, ma il trigger è un timer a interrupt e l'invio UART passa a HAL_UART_Transmit_DMA invece che bloccante.
- **01c**: qui entra in gioco l'auto-increment discusso sopra — nel timer callback fai una singola HAL_I2C_Master_Receive_DMA (dopo aver scritto il sub-address con MSB=1 se LIS2DE, o normale se LIS2DW con IF_ADD_INC=1) leggendo **3 byte** (LIS2DE) o **6 byte** (LIS2DW) in un colpo solo, poi nel callback HAL_I2C_MasterRxCpltCallback componi la stringa e la spedisci con HAL_UART_Transmit_DMA.

Se mi dici quale dei due chip è effettivamente montato sulla tua board (o mi mandi l'output del test di HAL_I2C_Master_Transmit), posso aiutarti a scrivere il codice di inizializzazione e lettura specifico invece che generico per entrambi.

Progetti

CapireLIS2DEoppureLIS2DW

```
void AccelerometerIdentitySearch(void){
	uint8_t buffer[2];
	char message[100];
	int n;

	// Leggiamo i due registri
	  st_de = HAL_I2C_Mem_Read(&hi2c1, LIS2DE_ADDR, WHO_AM_I, I2C_MEMADD_SIZE_8BIT,
&buffer[0], 1, HAL_MAX_DELAY);
	  st_dw = HAL_I2C_Mem_Read(&hi2c1, LIS2DW_ADDR, WHO_AM_I, I2C_MEMADD_SIZE_8BIT,
&buffer[1], 1, HAL_MAX_DELAY);

	  if(st_de == HAL_OK && buffer[0]==LIS2DE_ID){
		// è un LIS2DE
		n = sprintf(message, "WHO_AM_I=0x%02X, è un LIS2DE \r\n", buffer[0]);
		HAL_UART_Transmit(&huart2, (uint8_t*)message, n, HAL_MAX_DELAY);

	  } else if(st_dw == HAL_OK && buffer[1]==LIS2DW_ID){
		// è un LIS2DW
		n = sprintf(message, "WHO_AM_I=0x%02X, è un LIS2DW \r\n", buffer[1]);
		HAL_UART_Transmit(&huart2, (uint8_t*)message, n, HAL_MAX_DELAY);
	  } else{
		// Errore
		n = sprintf(message, "Errore \r\n");
		HAL_UART_Transmit(&huart2, (uint8_t*)message, n, HAL_MAX_DELAY);
	  }

}
```

Lab08_1a

Lab08_1a: leggi l'accelerazione misurata dall'accelerometer e mandala ad un remote terminal ogni 1s. Il valore dell'accelerazione deve essere espresso in g, con una precisione di 0.01g e segno corretto. Il FSR deve essere +/-2g. Sfrutta il while(1) e la HAL_Delay.

File .ioc

Pin

- PB8 = I2C1_SCL
- PB9 = I2C1_SDA

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
- SPI1: I2C: I2C

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LIS2DE_ADDR (0b0101000 << 1)
#define CTRL_REG1_ADDR (0x20)
#define CTRL_REG4_ADDR (0x23)
#define OUT_X_ADDR (0x29)
#define OUT_Y_ADDR (0x2B)
#define OUT_Z_ADDR (0x2D)
```

Private Variables

```
uint8_t ctrl1_config=0x17; // 0001 0111
// ODR[3:0] = 0001 -> 1 Hz
// lPen = 0 -> Normal Mode (non low-power)
// Zen, Yen, Xen = 1 -> tutti gli assi abilitati
uint8_t ctrl4_config = 0x00; // default 0000 0000
// FSR = +/-2g e Normal Mode (non Self Test)

uint8_t raw[5];
char message[100];
int n;

float acc_x, acc_y, acc_z;
```

Funzione di Inizializzazione

```
void AccelerometerInitialization(void){
      HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG1_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl1_config, 1, HAL_MAX_DELAY);
      HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG4_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl4_config, 1, HAL_MAX_DELAY);
}
```

Main

```
int main(void){
  AccelerometerInitialization();
  while (1){
        HAL_I2C_Mem_Read(&hi2c1, LIS2DE_ADDR, OUT_X_ADDR | 0x80, I2C_MEMADD_SIZE_8BIT,
raw, 5, HAL_MAX_DELAY);

        acc_x = ((int8_t)raw[0] * 15.6) / 1000.0;  // OUT_X
        acc_y = ((int8_t)raw[2] * 15.6) / 1000.0;  // OUT_Y (raw[1] è reserved, si
scarta)
        acc_z = ((int8_t)raw[4] * 15.6) / 1000.0;  // OUT_Z (raw[3] è reserved, si
scarta)

        n = sprintf(message, "X: %+.2f g\r\nY: %+.2f g\r\nZ: %+.2f g\r\n", acc_x,
acc_y, acc_z);
        HAL_UART_Transmit(&huart2, (uint8_t*)message, n, HAL_MAX_DELAY);

        HAL_Delay(1000);


  }
}
```

Lab08_1b

Lab08_1b: leggi l'accelerazione misurata dall'accelerometer e mandala ad un remote terminal ogni 1s. Il valore dell'accelerazione deve essere espresso in g, con una precisione di 0.01g e segno corretto. Il FSR deve essere +/-2g. Sfrutta timer interrupts e UART DMA.

File .ioc

Pin

- PB8 = I2C1_SCL
- PB9 = I2C1_SDA

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled
- SPI1:
  - I2C: I2C

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LIS2DE_ADDR (0b0101000 << 1)
#define CTRL_REG1_ADDR (0x20)
#define CTRL_REG4_ADDR (0x23)
#define OUT_X_ADDR (0x29)
#define OUT_Y_ADDR (0x2B)
#define OUT_Z_ADDR (0x2D)
```

Private Variables

```
uint8_t ctrl1_config=0x17; // 0001 0111
// ODR[3:0] = 0001 -> 1 Hz
// lPen = 0 -> Normal Mode (non low-power)
// Zen, Yen, Xen = 1 -> tutti gli assi abilitati
uint8_t ctrl4_config = 0x00; // default 0000 0000
// FSR = +/-2g e Normal Mode (non Self Test)

uint8_t raw[5];
char message[100];
int n;

float acc[3];
```

Funzioni

```
void AccelerometerInitialization(void){
	HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG1_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl1_config, 1, HAL_MAX_DELAY);
	HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG4_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl4_config, 1, HAL_MAX_DELAY);
	return;
}

void ReadAccelerometer(float *vector){
	HAL_I2C_Mem_Read(&hi2c1, LIS2DE_ADDR, OUT_X_ADDR | 0x80, I2C_MEMADD_SIZE_8BIT,
raw, 5, HAL_MAX_DELAY);

	vector[0] = ((int8_t)raw[0] * 15.6) / 1000.0;  // OUT_X
	vector[1] = ((int8_t)raw[2] * 15.6) / 1000.0;  // OUT_Y (raw[1] reserved)
	vector[2] = ((int8_t)raw[4] * 15.6) / 1000.0;  // OUT_Z (raw[3] reserved)
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
      ReadAccelerometer(acc);

      n = sprintf(message, "X: %+.2f g\r\nY: %+.2f g\r\nZ: %+.2f g\r\n", acc[0],
acc[1], acc[2]);

      HAL_UART_Transmit_DMA(&huart2, (uint8_t*)message, n);
    }
}
```

Inizializzazioni (Main, prima del while)

```
AccelerometerInitialization();
HAL_TIM_Base_Start_IT(&htim2);
```

Lab08_1c

Lab08_1c: leggi l'accelerazione misurata dall'accelerometer e mandala ad un remote terminal ogni 1s. Il valore dell'accelerazione deve essere espresso in g, con una precisione di 0.01g e segno corretto. Il FSR deve essere +/-2g. Sfrutta timer interrupts, UART DMA e I2C DMA.

File .ioc

Pin

- PB8 = I2C1_SCL
- PB9 = I2C1_SDA

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled
- SPI1:
  - I2C: I2C
  - DMA:
    - Request: RX (Peripheral to Memory)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA1 stream0 global interrupt = Enabled
    - I2C1 event interrupt = Enabled
    - I2C1 error interrupt = Enabled

Codice

Private Includes

```
#include "stdio.h"
#include "string.h"
```

Private Define

```
#define LIS2DE_ADDR (0b0101000 << 1)
#define CTRL_REG1_ADDR (0x20)
#define CTRL_REG4_ADDR (0x23)
#define OUT_X_ADDR (0x29)
#define OUT_Y_ADDR (0x2B)
#define OUT_Z_ADDR (0x2D)
```

Private Variables

```
uint8_t ctrl1_config=0x17; // 0001 0111
// ODR[3:0] = 0001 -> 1 Hz
// lPen = 0 -> Normal Mode (non low-power)
// Zen, Yen, Xen = 1 -> tutti gli assi abilitati
uint8_t ctrl4_config = 0x00; // default 0000 0000
// FSR = +/-2g e Normal Mode (non Self Test)

uint8_t raw[5];
char message[100];
int n;

float acc[3];
```

Funzioni

```
void AccelerometerInitialization(void){
	HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG1_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl1_config, 1, HAL_MAX_DELAY);
	HAL_I2C_Mem_Write(&hi2c1, LIS2DE_ADDR, CTRL_REG4_ADDR, I2C_MEMADD_SIZE_8BIT,
&ctrl4_config, 1, HAL_MAX_DELAY);
	return;
}


void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
      // Read the accelerometer
      HAL_I2C_Mem_Read_DMA(&hi2c1, LIS2DE_ADDR, OUT_X_ADDR | 0x80, I2C_ME-
MADD_SIZE_8BIT, raw, 5);
    }
}

void HAL_I2C_MemRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
    if (hi2c->Instance == I2C1)
    {
        acc[0] = ((int8_t)raw[0] * 15.6) / 1000.0;  // OUT_X
        acc[1] = ((int8_t)raw[2] * 15.6) / 1000.0;  // OUT_Y
        acc[2] = ((int8_t)raw[4] * 15.6) / 1000.0;  // OUT_Z

        n = sprintf(message, "X: %+.2f g\r\nY: %+.2f g\r\nZ: %+.2f g\r\n", acc[0],
acc[1], acc[2]);
        HAL_UART_Transmit_DMA(&huart2, (uint8_t*)message, n);
```

```
    }
}
```

Inizializzazioni (Main, prima del while)

```
AccelerometerInitialization();
HAL_TIM_Base_Start_IT(&htim2);
```